

# PROTOCOL

## STRK20 & PROTOCOL 01

### A TECHNICAL ANALYSIS

Protocol mechanics of the STRK20 privacy framework, established from the open-source [starkware-libs/starknet-privacy](#) repository, and a twelve-axis architectural comparison with the Protocol 01 privacy stack on Solana.

|                 |                |            |                   |
|-----------------|----------------|------------|-------------------|
| <b>12</b>       | <b>7</b>       | <b>2</b>   | <b>1</b>          |
| COMPARISON AXES | STARK CIRCUITS | PQ TARGETS | INTEGRATION POINT |

#### METHOD

Every STRK20 claim carries its source file in the public repository. Every Protocol 01 claim carries its `file:line`. No claim in this document rests on marketing material. Where a source could not be independently verified, it is marked as such rather than asserted.

#### TECHNICAL ANALYSIS

22 July 2026

Volta Team · Developed by Slashy Fx

# SCOPE & SUMMARY

STRK20 was open-sourced under Apache 2.0 on 8 July 2026. This document reads that source and compares the resulting architecture with Protocol 01, a STARK-based privacy stack on Solana.

### Primary sources for STRK20

- starkware-libs/starknet-privacy — Cairo pool contract, Rust discovery service, TypeScript SDK, Lean 4 proofs
- SNIP-36 — in-protocol proof verification
- @starknet-io/types-js@0.10.3 — Wallet API
- stwo / stwo-cairo — the prover
- StarkWare post-quantum roadmap, 30 June 2026
- OpenZeppelin privacy contracts audit

### Primary sources for Protocol 01

- stark/src/compact.rs — the proof protocol actually verified on-chain
- programs/p01\_stark\_verifier/ — the on-chain verifier
- programs/zk\_shielded/ — the shielded pool
- packages/specter-sdk — recipient discovery, published on npm
- programs/specter/ — the announcement transport

## What the comparison finds

### The two systems solve the same problem with opposite primitives

STRK20 replaces the anonymity set of a Merkle accumulator with the pseudorandomness of storage addresses, and enforces value conservation with an on-chain ledger rather than a circuit constraint. Protocol 01 uses a Merkle tree with fixed denominations, so value is never a circuit input. Neither system places value conservation inside an AIR. Both are transparent and hash-based, with no trusted setup.

### The complementarity is genuine and bidirectional

STRK20 recipient discovery is bucketed per recipient, so scan cost scales with a user's own activity rather than with pool volume — architecturally stronger than Protocol 01 on that axis. In exchange, opening a channel publicly reveals the recipient's address, and recipients must pre-register. Protocol 01's stealth-address derivation requires no recipient action and leaks no recipient address, but its scan cost scales with total pool volume. Each design solves the other's structural weakness.

### The post-quantum surface is asymmetric

Both proof systems are hash-based and survive Shor. StarkWare's June 2026 PQ roadmap addresses hashing and commitments. It does not address how the encryption layer shipped in STRK20 — encrypted balances, and the on-chain encrypted viewing key a designated auditor can decrypt — would itself migrate to post-quantum primitives. Section 4 analyses that gap precisely and proposes a surgical remedy.

| SECTION | PAGES | CONTENTS                                                                                                                            |
|---------|-------|-------------------------------------------------------------------------------------------------------------------------------------|
| 1       | 3–10  | STRK20 mechanics: note model, encryption, addressing, transaction phases, proof path, proving system, compliance, developer surface |
| 2       | 11–12 | Protocol 01 stack: proving parameters, circuits, pool, recipient discovery                                                          |
| 3       | 13–17 | The twelve-axis comparison                                                                                                          |
| 4       | 18    | The post-quantum gap                                                                                                                |
| 5       | 19    | Proposed integration point                                                                                                          |
| 6       | 20    | Sources and verification caveats                                                                                                    |

The central architectural surprise, and the one that inverts the intuition of anyone arriving from Tornado, Zcash, or a denominated-pool design.

### There is no Merkle tree

No accumulator, no root, no historical-roots window. A repository-wide search for merkle returns a single file, and that hit concerns Juno's own state trie rather than the protocol. Notes live directly in Cairo storage maps.

```
notes: Map<felt252, Note>,
nullifiers: Map<felt252, bool>,
recipient_channels: Map<ContractAddress, Vec<EncChannelInfo>>,
```

packages/privacy/src/privacy.cairo:88-99

### A note is not a hash commitment

```
pub struct Note {
    /// The packed value of the note `(salt, amount)`
    pub packed_value: felt252,
    /// The token address of the note (zero for encrypted notes).
    pub token: ContractAddress,
}
```

packages/privacy/src/objects.cairo:89-100

Owner, token and index are not part of a commitment preimage. They are bound **implicitly by the storage key**.

```
note_id = Poseidon(NOTE_ID_TAG, channel_key, token, index, 0)
nullifier = Poseidon(NULLIFIER_TAG, channel_key, token, index, 0, owner_private_key)
```

packages/privacy/src/ hashes.cairo:200-236 — all protocol hashing is Poseidon over the STARK field, with 17 domain-separation tags, all at :V1

### The consequence

Because channel\_key is a secret shared only between sender and recipient, storage slots are pseudorandom to any observer. **Storage-address pseudorandomness substitutes for the anonymity set of a Merkle accumulator.** This is a different security assumption, not a weaker or stronger one — and it is the reason the post-quantum analysis in Section 4 lands differently here than it would against a commitment tree.

### Spending: a storage read, not a membership proof

```
let subchannel_marker = compute_subchannel_marker(...);
assert(self.subchannel_exists.read(subchannel_marker), errors::SUBCHANNEL_NOT_FOUND);
let packed_value = self.notes.entry(note_id).packed_value.read();
assert(packed_value.is_non_zero(), errors::NOTE_NOT_FOUND);
```

packages/privacy/src/privacy.cairo:585-629

### Nullifier asymmetry

The nullifier includes the **owner's** private key. A sender therefore cannot compute the nullifier of a note they created, and cannot observe when their payment is later spent. This is a property Protocol 01 does not have, and it is an elegant one.

No KEM in the modern sense, and no authenticated symmetric cipher. Two layers: elliptic-curve Diffie-Hellman on the STARK curve, then additive masking.

### Layer 1 – channel opening (asymmetric)

ECDH on the STARK curve with a per-channel ephemeral scalar. Only the x-coordinate of the shared point is used as key material. The KDF is a single domain-separated Poseidon hash, and the cipher is field addition.

```
fn _compute_shared_x(ephemeral_secret: felt252, public_key: felt252) → (felt252, felt252) {
    let ephemeral_pub_point = GEN_P().mul(scalar: ephemeral_secret);
    let public_point = EcPointTrait::new_from_x(x: public_key)...;
    let shared_point = public_point.mul(scalar: ephemeral_secret);
    let shared_x = ...x();
}
```

packages/privacy/src/utils.cairo:123-145

```
enc_channel_key = h(CHANNEL_KEY_TAG, rK.x) + channel_key
enc_sender_addr = h(SENDER_ADDR_TAG, rK.x) + sender_addr
```

### Layer 2 – in-channel data (symmetric)

```
enc_amount = (Poseidon(ENC_AMOUNT_TAG, channel_key, token, index, 0, salt) + amount) mod 2^128
enc_token = Poseidon(ENC_TOKEN_TAG, channel_key, index, 0, salt) + token
packed_value = salt * 2^128 + enc_amount // 120-bit sender-chosen salt
```

packages/privacy/src/ hashes.cairo:212-222

### Channel key derivation

```
channel_key = Poseidon(CHANNEL_KEY_TAG, sender_addr, sender_private_key,
                      recipient_addr, recipient_public_key)
```

Channels are strictly **directional**: Alice→Bob and Bob→Alice are two separate channels. A deposit is a channel from an account to itself.

#### Canonical key constraint

Because only x-coordinates are used,  $k$  and  $ORDER-k$  would be indistinguishable. The contract therefore requires  $is\_canonical\_key(key) = key < HALF\_ORDER$ . An out-of-range key compiles but silently derives wrong channel keys, and notes sent to that account never decrypt.

#### The scheme is fixed in the contract

There is no scheme or version field on Note or EncChannelInfo. The only versioning is the :V1 suffix inside the Poseidon domain tags. Changing the scheme requires a contract upgrade through ReplaceabilityComponent. The encryption layer is **not pluggable** at the SDK level.

#### OpenZeppelin audit — the ephemeral scalar

The audit notes that the ephemeral scalar is client-supplied and validated only as non-zero: the contract “does not enforce uniqueness or prevent reuse of  $r$ ”, creating a risk of deterministic ciphertext reuse. A second finding is that an invalid `auditor_public_key` triggers `EcPointTrait::new_from_x(...).unwrap()` panics able to block registration and withdrawal system-wide. Audited commit c5e2fb53...

How a sender tells a recipient that a note is theirs. This is the axis on which the two systems diverge most sharply.

There is **no detection tag**, **no view tag**, and **no full-pool trial decryption**. The contract keeps one bucket per recipient, keyed by the recipient's **plaintext Starknet address**.

```
/// Map of recipient_addr to a list of their encrypted channels.  
recipient_channels: Map<ContractAddress, Vec<EncChannelInfo>>,
```

packages/privacy/src/privacy.cairo:88-89

The sender appends an `EncChannelInfo` into the recipient's bucket through the `Append server` action. The recipient reads only their own bucket and trial-decrypts each entry.

### The linkability this creates

The storage slot is derived deterministically from the plaintext address via `get_storage_var_address("recipient_channels", [recipient_address])`, so any observer watching state diffs can see that a specific address has just received a new channel.

Only the **first** transfer of a given (sender, recipient) pair does this; subsequent notes land in slots derived from the secret `channel_key`. The SDK raises a `WarningCode.USER_LINKAGE` when more than one `OpenChannel` appears in a single transaction, and the documentation lists channel-open linkability as a known limitation, alongside the acknowledgement that "edges are public by design".

### Both parties must be registered

```
let sender_public_key = self.public_key.read(sender_addr);  
assert(sender_public_key.is_non_zero(), errors::SENDER_NOT_REGISTERED);  
let recipient_public_key = self.public_key.read(recipient_addr);  
assert(recipient_public_key.is_non_zero(), errors::RECIPIENT_NOT_REGISTERED);
```

packages/privacy/src/privacy.cairo:368-379

A private transfer therefore cannot target an address derived on the fly. The documented workaround for unregistered recipients is an escrow contract holding funds behind `poseidon(ESCROW_COMMITMENT_TAG, secret)`, with the secret shared off-chain as a claim link.

### Discovery: a three-level dense index walk

| LEVEL | OPERATION                                                                                                  |
|-------|------------------------------------------------------------------------------------------------------------|
| 1     | Read own <code>recipient_channels</code> , ECDH trial-decrypt each entry                                   |
| 2     | Walk subchannel indices 0, 1, 2... per channel, unmasking the token                                        |
| 3     | Walk note indices per (channel, token), unmask amounts, test each derived nullifier against on-chain state |

The walk terminates at the first empty slot. All slots are `WriteOnce` and indices must be sequential (`INDEX_NOT_SEQUENTIAL`); density is what guarantees termination.

### The architectural advantage

**Scan cost is proportional to a user's own activity, not to pool volume.** "A pool with millions of transfers costs you no more to scan than a quiet one." Discovery is a free read: no proof, no fee, no submission. Notes become spendable ten blocks after creation.

Not a shield / transfer / unshield triple, but a list of actions grouped into eight strictly ordered phases.

| PHASE | ACTION                         | ROLE                                                       |
|-------|--------------------------------|------------------------------------------------------------|
| 0     | SetViewingKey                  | Registration. Immutable once set (WriteOnce)               |
| 1     | OpenChannel                    | Creates the directional sender → recipient channel         |
| 2     | OpenSubchannel                 | One subchannel per token within a channel                  |
| 3     | Deposit                        | Public funds enter the pool. Always to self                |
| 4     | UseNote                        | Consumes a note, writes its nullifier                      |
| 5     | CreateEncNote / CreateOpenNote | Creates output notes                                       |
| 6     | Withdraw                       | Funds exit to a public address                             |
| 7     | InvokeExternal                 | Calls an anonymizer contract. At most once per transaction |

packages/privacy/src/actions.cairo:275-316 – assert(action\_phase ≥ curr\_phase, errors::ACTIONS\_OUT\_OF\_ORDER)

#### SHIELD

Deposit + CreateEncNote to self

#### TRANSFER

UseNote + CreateEncNote(recipient) +  
CreateEncNote(change)

#### UNSHIELD

UseNote + Withdraw

### Value conservation is not a circuit constraint

It is a per-token temporary balance ledger inside a single transaction. Deposit and UseNote add; CreateNote and Withdraw subtract. The balance may never go negative, and every token must end at exactly zero.

```
fn subtract_balance(...) {
    let new_value = current_balance.checked_sub(amount)
    .expect(errors::NEGATIVE_INTERMEDIATE_BALANCE);
}
fn assert_valid(self: SquashedTokenBalances) {
    for (_token, _, balance) in self.into_entries() {
        assert(balance.is_zero(), errors::FINAL_BALANCE_MUST_BE_ZERO);
    }
}
```

packages/privacy/src/objects.cairo:9-27

A direct point of comparison: STRK20 solves conservation with an on-chain state machine, Protocol 01 solves it with pool denomination. **Neither system places it inside an AIR.**

### Open notes — a deliberate plaintext escape hatch for DeFi

With salt = OPEN\_NOTE\_SALT (=1) the token address and amount are stored in the clear, and the amount is written after proof generation by an anonymizer contract. Encrypted notes always use salt ≥ 2; salt = 0 means the note does not exist. “Open notes trade amount privacy for that late binding.”

### DeFi composability through anonymizer contracts

A helper exposes a single mandatory entrypoint privacy\_invoke, which asserts that the caller is the pool. The pool withdraws tokens to the helper, calls it, and the helper returns OpenNoteDeposit { note\_id, token, amount } instructions for whatever should be credited back into private notes. The ExternalContractInvoked event exposes the target address and selector but not the calldata. Reference packages exist for Ekubo swaps and Vesu lending.

Where a client-generated privacy proof is actually verified. This is the largest architectural divergence from an on-chain verifier design.

| # | STEP                                                                                                                                                                                                                                  |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | The SDK compiles the client actions and sends them to an <b>operator-run proving service</b>                                                                                                                                          |
| 2 | The service executes them in a <b>virtual Starknet block</b> (Virtual SNOS), a modified OS with reduced functionality. The <code>add_mod</code> , <code>mul_mod</code> and Keccak builtins are excluded from proven claims in phase 1 |
| 3 | The Stwo proof travels in the <b>optional</b> proof (uint32[]) and proof_facts (felt252[]) fields of an Invoke V3 transaction — <b>not in calldata</b>                                                                                |
| 4 | An <b>S-two verifier written in Rust, embedded in the gateway/sequencer</b> , verifies it. Not a Cairo contract, and not (in phase 1) recursed into the L2 state proof settled on Ethereum                                            |
| 5 | The pool contract reads only the small proof_facts through a dedicated syscall, and validates the program variant, base block, validity window and message hash                                                                       |

SNIP-36 — [community.starknet.io/t/snip-36-in-protocol-proof-verification/116123](https://community.starknet.io/t/snip-36-in-protocol-proof-verification/116123)

### Two properties StarkWare states explicitly

*“applications built with this feature will have **degraded security** compared to native Starknet applications” — to be mitigated in a follow-up release.*

*“The protocol is **not zero-knowledge by default**; proofs reveal no execution data beyond declared public input.”*

In phase 1 proofs are verified by consensus but not by the SNOS, are not included in blocks, are not retrievable from the feeder gateway, and are retained only for a finite period. Full nodes do not re-verify. The v0.14.2 privacy model is described as de facto rather than formally zero-knowledge, with full ZK deferred to phase 2.

### Why SNIP-36 exists

*“Including the full proof (often tens of thousands of felts) in calldata exceeds the present transaction limit (5K felts).”*

In-contract STARK verification on Starknet does exist — Herodotus's Integrity verifier — but had to be split across multiple contracts and transactions because of classhash size limits, calldata caps and the 3M Cairo step limit per transaction. StarkWare's response was to move verification into the protocol rather than to optimise a contract verifier. That option is available to an L2 designing its own sequencer; it is not available to an application deployed on an existing L1.

### Proof validity window

The contract rejects proofs older than `proof_validity_blocks` (450 blocks, roughly 15 minutes at 2 s/block). SDK convention is to prove at `currentBlock - 10`. Any on-chain state the proof reads — account viewing key, depositor balance, nullifier set — must have been written at least ten blocks before the base block. This is the functional analogue of a historical-roots window, though it solves proof freshness rather than concurrency on tree state.

Stwo, a Circle STARK over M31 — and the architectural point that explains everything downstream: Cairo is a fixed CPU AIR.

**The field: M31,  $p = 2^{31} - 1$**

Tower M31 → CM31 (quadratic) → QM31 (quartic, used for Fiat-Shamir and DEEP/FRI randomness).

**Why not a 64-bit field**

M31 has 2-adicity of exactly 1 ( $p-1 = 2 \cdot (2^{30}-1)$ ): no smooth  $2^k$  subgroup for a radix-2 FFT.

The Circle STARK construction works around this using the circle curve  $x^2 + y^2 = 1$ , whose group of points has order  $p+1 = 2^{31}$ , an exact power of two. The circle FFT keeps the same radix-2 butterfly network, operating on bivariate polynomials restricted to the circle. The construction is due to Haböck, Levit and Papini, ePrint 2024/278, reporting a 1.4× speed-up over a BabyBear STARK.

The practical argument is SIMD lane density: reduction mod  $2^{31}-1$  is a cyclic shift rather than a division, and the prover's hot paths vectorise into 32-bit AVX2, AVX-512, NEON and WASM-SIMD lanes.

|                    |                                                                                                                                                                        |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commitment hash    | Blake2s-256 by default; Poseidon252 optionally, for Ethereum settlement                                                                                                |
| Merkle tree        | "lifted": a node hash depends on both the children hashes <b>and</b> the column values injected at that layer, committing columns of different heights without padding |
| Default parameters | log_blowup = 1 (blowup 2), 70 queries, 26 grinding bits, plus an interaction-phase proof-of-work                                                                       |
| Target soundness   | <b>96 bits conjectured</b> (list-decoding regime), not proven under the Johnson bound                                                                                  |
| Trusted setup      | none — "fully transparent, no toxic waste, no MPC ceremony"                                                                                                            |
| Zero-knowledge     | not ZK by default                                                                                                                                                      |

**The decisive architectural point**

Cairo is a **single fixed CPU AIR**. One immutable set of polynomial constraints proves valid execution of the Cairo machine. The application program is not compiled into constraints; it is **data** placed in public memory and consumed as trace.

*"We describe a single set of polynomial equations for the statement that the execution of a program on this architecture is valid. Given a statement one wishes to prove, Cairo allows writing a program that describes that statement, instead of writing a set of polynomial equations." — Cairo paper, ePrint 2021/1063*

In practice stwo-cairo decomposes this into 69+ components, one per opcode and per builtin, linked by LogUp lookup arguments. Builtins are memory-mapped peripherals that replace expensive in-VM computation with dedicated AIR components. This is what "the circuit and the contract are the same code" means concretely: no circuit DSL, no per-application AIR authoring, no per-application verifier deployment.

| Official S-two benchmark — 48-core EPYC, CPU only |         |            |              |
|---------------------------------------------------|---------|------------|--------------|
| WORKLOAD                                          | PROVING | PROOF SIZE | VERIFICATION |
| Fibonacci $n = 2^{20}$                            | 18.61 s | 808 KB     | 414 ms       |
| SHA-256 $n = 2^{18}$ bytes                        | 121 s   | 1315 KB    | 83 ms        |



A complete, formally specified compliance layer — and, as Section 4 argues, the densest harvest-now-decrypt-later surface in the system.

|                     |                                                                                                                                                                                                                                     |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Auditor key         | a <b>single</b> <code>auditor_public_key</code> : felt252 in contract storage, set at deployment, changeable by the <code>security_governor</code> role. Documentation describes threshold-key support; the contract stores one key |
| Key escrow          | at registration the contract itself encrypts the user's <b>private</b> viewing key to the auditor key, using ephemeral ECDH plus a Poseidon mask, stored on-chain and immutable, with a public <code>ViewingKeySet</code> event     |
| Disclosure scope    | <b>complete bidirectional history</b> : all incoming and outgoing channels, every note amount, nullifier-to-note matching, and forward and backward tracing                                                                         |
| Auditor powers      | viewing only, never spending — spending requires an account signature verified inside the proof                                                                                                                                     |
| Formal verification | ~60 Lean 4 files covering notes, channels, subchannels, no-replay, spendability, compliance and tracing                                                                                                                             |

### Deposit screening is enforced on-chain

Since v0.14.3 every deposit carries a SNIP-12 `DepositorValidation { depositor, issued_at }` attestation signed by an off-chain screener whose STARK public key is in contract storage; the pool verifies the ECDSA signature. Attestations expire after 300 seconds with 60 seconds of tolerated future skew.

Running a self-hosted prover does not bypass this. One documentation discrepancy is worth noting: the public docs name “FPI” as the screening provider, while the repository's `elliptic-proxy` calls Elliptic's AML API. Operator allow and deny lists (`blockOverrideAddresses`, `additionalBlockedAddresses`) also exist.

## The phase-1 trust model, as documented

Both of the following are explicit design choices recorded in StarkWare's own specifications, not defects. They are included here because any comparison of privacy guarantees has to state where trust sits.

### The private viewing key is sent to the proving service

```
const executeViewCalldata = callDataCompiler.compile("compile_actions", [
  userAddress, user.viewingKey, cairoActions
]);
```

`sdk/src/internal/proof-invocation-factory.ts:128-140`

A hosted prover therefore learns the key that decrypts the user's entire history.

### The discovery service receives decryption keys and returns decrypted data

*“The service decrypts and returns decrypted channels and unspent notes. Decryption keys must be provided per request.” — “The operator can observe: which recipients are active and when. How many channels, subchannels, and notes each recipient has. Token addresses used per channel. Timing of sync activity.”*

`crates/discovery-service/specs/05-security-considerations.md`

OHTTP, based on X25519 with an optional relay, mitigates IP correlation only.

The Privacy Wallet API is Starknet Wallet API v0.10.3, shipped as TypeScript types. It exposes exactly three STRK20 methods.

```
wallet_strk20InvokeTransaction // { actions: STRK20_ACTION[] } → { transaction_hash }
wallet_strk20PrepareInvoke    // { actions, simulate? } → STRK20_CALL_AND_PROOF
wallet_strk20Balances         // { tokens: Address[] } → STRK20_BALANCE_ENTRY[]
```

@starknet-io/types-js@0.10.3 — dist/types/wallet-api/methods.d.ts:135,157,174

### The four action variants

**deposit** (always to self, no recipient field), **withdraw**, **transfer** (accepts the literal "OPEN" as amount), **invoke**. Errors include a dedicated **PRIVACY\_LEAK** alongside **NOT\_REGISTERED** and **INSUFFICIENT\_PRIVATE\_BALANCE**.

### No note scanning in the Wallet API

A dapp reads aggregate per-token balances only. Scanning exists solely in the lower-level Privacy SDK (`discoverNotes`, `discoverChannels`), which requires holding the viewing key. This is deliberate: "A normal dapp should not receive the user's viewing key."

## The lower-level SDK — a fluent builder

```
await transfers.build(opts)
  .with(TOKEN, t => t.inputs(note).transfer({ recipient, amount }).withdraw({ amount }))
  .surplusTo(self)
  .execute({ provingBlockId });
```

Verbs are `register`, `deposit`, `transfer`, `withdraw`, `setup`, `invoke` — not `shield` / `unshield` / `claim`.

|                  |                                                                                                                                                                                        |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dapp requirement | <code>starknet.js</code> ≥ 10.4.0 with <code>WalletAccountV6</code> . The npm latest tag is 10.0.2; STRK20-capable releases sit on next, so the version must be pinned explicitly      |
| Prover location  | <b>server-side</b> , neither in the wallet nor in the dapp. Self-hosting is supported through a published container image; StarkWare describes phase-1 proving services as self-hosted |
| Proving time     | published figures conflict and are not reconciled by any source: ~29 s (12-core / 46 GiB) in the pipeline diagram, ~4 s in the SDK README, "few-seconds area" per the SNIP-36 author   |
| Wallets          | Ready is live dapp-facing; Xverse has in-wallet privacy with dapp-facing API support in progress. Capabilities should be detected at runtime                                           |
| Fees             | fixed per transaction, 4 STRK announced — "closer to a gas fee than a toll"                                                                                                            |
| Deployment       | live on mainnet and Sepolia. Mainnet pool at <code>0x040337b1af...812a</code>                                                                                                          |

### Account abstraction cuts both ways

Spend authorisation reuses the existing account's `is_valid_signature`, so multisig, hardware and smart accounts work without protocol changes. The cost is that a receiving account must be deployed and finalised roughly ten blocks before `register()` can be proven, because the prover reads the account's viewing-key slot at the proof base block.

A hand-written STARK protocol over Goldilocks, verified by a Solana program. Figures below are read from the code paths actually exercised on-chain.

|                           |                                                                                                                                                                                                                                                                                                                   |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Library                   | Winterfell 0.10.3 is used as a field-arithmetic and AIR-specification library only. The proof format verified on-chain is a hand-rolled protocol in <code>stark/src/compact.rs</code> , importing only <code>BaseElement</code> and <code>FieldElement</code> . The on-chain program has no Winterfell dependency |
| Field                     | Goldilocks, $p = 2^{64} - 2^{32} + 1$ , confirmed independently on both prover and verifier                                                                                                                                                                                                                       |
| Merkle / Fiat-Shamir hash | <b>SHA-256</b> . Chosen over Blake3 because the <code>sol_blake3</code> syscall sits behind a runtime feature inactive on devnet and mainnet, making software Blake3 cost ~15k CU per hash; <code>sol_sha256</code> costs ~85 CU per call                                                                         |
| Note-tree hash            | Poseidon over Goldilocks, $t=3$ , 30 full rounds, S-box $x^7$ , circulant MDS — deliberately distinct from the SHA-256 used for the proof's own Merkle commitments                                                                                                                                                |
| FRI parameters            | blowup 16; 27 queries for circuits 0/1/2/4 and 22 for 3/5/6; 16 grinding bits; FRI folding factor 2; final polynomial 16 coefficients; base field only, no extension                                                                                                                                              |
| Claimed soundness         | 124 bits (27 queries) or 104 bits (22 queries) by the queries $\times \log_2(\text{blowup}) + \text{grinding heuristic}$ , described in code as classical soundness                                                                                                                                               |
| Trusted setup             | none. An exhaustive search for elliptic-curve, pairing or ceremony code across the prover and verifier returns only prose asserting their absence                                                                                                                                                                 |

## Two caveats stated for symmetry with the STRK20 figures

First, the soundness formula above is the naive heuristic, without a proximity-gaps discount, and the code does not distinguish conjectured from proven soundness the way `stwo-cairo`'s documentation does. Second, the out-of-domain point is sampled from the **base field** ( $\sim 2^{64}$ ) rather than an extension, which bounds the Schwartz-Zippel term well below the quoted figure. The honest statement is that the quoted number is the FRI query and grinding term alone.

## Two-phase on-chain verification

Phase 1 performs the OOD range check, transcript re-derivation, grinding and query-position derivation, trace and quotient Merkle openings, FRI fold consistency, per-query transition checks and boundary constraints. Phase 2 performs the DEEP-ALI identity at the out-of-domain point across the full constraint set.

The split exists because only about 24% of blowup-16 query positions land on trace-aligned rows, leaving the remaining 76% unconstrained without an out-of-domain random-linear-combination check — and because a single-transaction verification exceeded Solana's compute ceiling. Two devnet signatures consuming the full 1,400,000 units are cited in the source as the evidence that forced the split. The two phases run as **two separate transactions**, since 1,400,000 is a per-transaction limit.

## Measurement caveat

The repository contains no committed compute-unit measurements. The only enforcement is an assertion that each phase stays below 1,400,000 units, with actual values printed at test runtime and not recorded. Design-note estimates exist in comments but are not measurements, and are not quoted as such in this document.

| ID | CIRCUIT              | WIDTH | LENGTH | LDE  | DEPTH | QUERIES | CONSTR. | PROOF SIZE |
|----|----------------------|-------|--------|------|-------|---------|---------|------------|
| 0  | subscriber_ownership | 3     | 32     | 512  | 9     | 27      | 3       | 79,993 B   |
| 1  | denominated_pool     | 3     | 128    | 2048 | 11    | 27      | 4       | 120,665 B  |
| 2  | balance_proof        | 4     | 128    | 2048 | 11    | 27      | 7       | 121,113 B  |
| 3  | merkle_path          | 6     | 512    | 8192 | 13    | 22      | 11      | 138,293 B  |
| 4  | confidential_balance | 4     | 256    | 4096 | 12    | 27      | 10      | 144,041 B  |
| 5  | transfer             | 7     | 512    | 8192 | 13    | 22      | 28      | 138,661 B  |
| 6  | merkle_update        | 10    | 512    | 8192 | 13    | 22      | 19      | 139,765 B  |

Read from `CircuitConfig` in `programs/p01_stark_verifier/src/compact_proof.rs:50-166`, the authority the on-chain parser uses

Range checks are not present in the `balance_proof` AIR and value conservation is not present in the `confidential_balance` AIR; both are enforced on-chain instead. The transfer circuit carries a value-conservation accumulator column.

## The pool

|                    |                                                                                                                                                                                                                                             |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tree               | Poseidon-Goldilocks Merkle tree, depth 15 (32,768 notes per pool), 100 historical roots, on-chain subtree maintenance via circuit 6                                                                                                         |
| Denominations      | SOL: 0.1, 1, 10, 100, 500, 1000 · USDC: 1, 10, 100, 1000, 10k, 20k, 50k — one pool per (token, denomination) pair                                                                                                                           |
| Commitment         | Poseidon(nullifier, epoch_hash), with nullifier = Poseidon(preimage, secret) and epoch_hash = Poseidon(epoch, mint)                                                                                                                         |
| Value conservation | structural: the value is the pool denomination, never a circuit input                                                                                                                                                                       |
| Proof transport    | uploaded into a program-owned buffer in 145 chunk transactions of 1,000 bytes, after 14 resize transactions of 10,240 bytes. All proofs are padded to a uniform 145,000 bytes so that neither proof size nor circuit identity is observable |
| Status             | devnet, not mainnet                                                                                                                                                                                                                         |

## Recipient discovery (Specter)

Discovery is a **hybrid X25519 + ML-KEM-768 construction**. The combiner is OR-secure: the shared secret is a KDF over the concatenation of both secrets, so discovery confidentiality holds if either scheme holds. An adversary able to break X25519 still cannot recover the shared secret without the ML-KEM-768 private key.

|                        |                                                                                                       |
|------------------------|-------------------------------------------------------------------------------------------------------|
| <code>r</code>         | <code>= ephemeral X25519 keypair</code>                                                               |
| <code>s_c</code>       | <code>= X25519(r, V)</code>                                                                           |
| <code>(ct, s_q)</code> | <code>= ML-KEM-768.Encaps(K)</code>                                                                   |
| <code>s</code>         | <code>= HKDF-SHA256(ikm = s_c    s_q, info = "p01-hybrid-stealth-v2"    SHA256(R    ct), L=32)</code> |
| <code>viewTag</code>   | <code>= SHA256(s)[0]</code>                                                                           |
| <code>seed</code>      | <code>= HKDF-SHA256(ikm = s, salt = spendingPubKey, info = "p01-stealth-seed", L=32)</code>           |
| <code>stealth</code>   | <code>= Ed25519_keygen_from_seed(seed)</code>                                                         |

Unlike ERC-5564 or Monero there is no curve point addition: the stealth address is a fresh Ed25519 key derived from an HKDF seed, and the recipient's spending public key is used only as HKDF salt. ML-KEM-768 sizes are 1184-byte public key, 2400-byte secret key, 1088-byte ciphertext. The 1088-byte ciphertext is carried on-chain in a program-owned account in two chunks. Legacy non-KEM meta-addresses are rejected by default at every entry point. The layer is published as `@protocol-01/specter-sdk` on public npm. Note that while **discovery** is quantum-resistant, the stealth address itself is an Ed25519 key, so the spending path is not.

### 3.1 Proof system and field

|                   | STRK20                                | PROTOCOL 01                                                |
|-------------------|---------------------------------------|------------------------------------------------------------|
| System            | Stwo, Circle STARK                    | Hand-written compact protocol, Winterfell as field library |
| Field             | M31, $p = 2^{31}-1$ , CM31/QM31 tower | Goldilocks, $p = 2^{64}-2^{32}+1$ , no extension           |
| Structural device | circle curve, order $p+1 = 2^{31}$    | native smooth subgroup                                     |
| Commitment hash   | Blake2s-256, or Poseidon252           | SHA-256 (Solana syscall constraint)                        |
| Merkle            | lifted (node = children + columns)    | standard                                                   |
| Blowup            | 2                                     | 16                                                         |
| Queries           | 70                                    | 27 or 22                                                   |
| Grinding          | 26 bits + interaction PoW             | 16 bits                                                    |
| Stated soundness  | 96 bits conjectured                   | 124 / 104 bits, classical heuristic                        |
| OOD point         | QM31 (quartic extension)              | base field $\sim 2^{64}$                                   |
| Trusted setup     | none                                  | none                                                       |
| Zero-knowledge    | not ZK by default                     | no ZK masking either                                       |

#### Reading the soundness figures fairly

STRK20 uses blowup 2 with 70 queries, which reaches 96 bits only under a conjectured list-decoding assumption. Protocol 01 uses blowup 16, more conservative per query, but samples the out-of-domain point from the base field rather than an extension, which bounds its Schwartz-Zippel term well below its stated figure. **Neither number should be quoted without naming its regime.**

### 3.2 Constraint authoring model

This is the structural difference from which most others follow. STRK20 has one fixed CPU AIR and the program is data; writing a new privacy feature means writing Cairo. Protocol 01 has one hand-written AIR per circuit.

#### The maintenance cost of hand-written AIRs, measured

Changing a single transition constraint on one circuit requires editing four hand-written sites plus one generated table:

1. the AIR itself, plus its constraint-degree list and periodic columns
2. the prover-side quotient computation
3. the verifier's out-of-domain evaluator
4. the verifier's **per-query re-check**, an independent hand-written copy of the same logic
5. the precomputed periodic coefficients, if the periodic columns change

A boundary-constraint change adds two further mirrored sites with no cross-check test between them. The only automated guard that exists covers the periodic constants: seven parity tests re-run the prover's inverse NTT and compare against the verifier's baked tables. **This is the clearest advantage of the Cairo model, and it is worth stating plainly.**

### 3.3 Verification location and cost

|                    | STRK20                                                                                              | PROTOCOL 01                                            |
|--------------------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Verifier           | S-two in Rust, inside the gateway / sequencer                                                       | Solana program, on-chain                               |
| Compute ceiling    | none; the constraint is <b>economic</b> (125 L2gas per byte propagation + 5 storage + a 10M buffer) | <b>1,400,000 CU per transaction, hard</b>              |
| Splitting          | none                                                                                                | two transactions (phase 1, then DEEP-ALI)              |
| Inherited security | <b>phase 1: verified by consensus, not by the SNOS</b>                                              | <b>full L1 security, re-verified by all validators</b> |
| Contract role      | reads proof_facts only                                                                              | parses the entire proof                                |

#### The trade-off, stated symmetrically

STRK20 has no per-proof compute ceiling **because verification was moved out of the contract layer**, and StarkWare documents that phase-1 applications inherit degraded security until recursion into the SNOS lands. Protocol 01 pays 1.4M compute units **because its proof is verified by the chain itself**, with no additional trust assumption. These are two positions on the same trade-off rather than a ranking. Changing the L2 protocol is available to a team that owns the sequencer; it is not available to an application on an existing L1.

### 3.4 Proof transport

|             | STRK20                                                                                              | PROTOCOL 01                                                                           |
|-------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Size        | not published for STRK20; S-two generally 808 KB to 1315 KB; SNIP-36's pricing model assumes 500 KB | <b>80 KB to 144 KB, padded to a uniform 145,000 B</b>                                 |
| Path        | proof / proof_facts fields of Invoke V3, outside calldata                                           | 145 chunk transactions of 1,000 B into a buffer account, after 14 resize transactions |
| Persistence | not in blocks, not at the feeder gateway, retained a few weeks                                      | buffer account, rent locked, closed after use                                         |
| Rent        | none                                                                                                | ~0.87 SOL transient float                                                             |

#### A genuine asymmetry in Protocol 01's favour

**The Protocol 01 proof is five to ten times smaller.** Goldilocks with blowup 16 and 22–27 queries yields a substantially more compact proof than M31 with blowup 2 and 70 queries. The cost is distributed differently: STRK20 absorbs transport in the protocol, Protocol 01 absorbs it in 159 transactions and locked rent.

#### Why uniform proof size

Padding every proof to 145,000 bytes closes two metadata leaks: proof size as a circuit fingerprint, and — paired with a nonce-seeded buffer account — circuit identity at initialisation. The headroom above the largest circuit is 959 bytes, which is a tight margin and is monitored as such.

## 3.5 Note model

|              | STRK20                                            | PROTOCOL 01                                   |
|--------------|---------------------------------------------------|-----------------------------------------------|
| Structure    | { packed_value: felt252, token: ContractAddress } | Poseidon commitment                           |
| Identity     | storage address                                   | leaf in a Merkle tree                         |
| Amount       | masked by hash-and-add, 120-bit salt              | the pool denomination, never a circuit input  |
| Anonymity    | storage-slot pseudorandomness                     | Merkle tree anonymity set                     |
| Multi-asset  | one pool for every ERC-20                         | one pool per (token, denomination) — 13 pools |
| Conservation | in-transaction on-chain ledger                    | denomination + accumulator column             |

### The substantive trade

Denominated pools are structurally safe with respect to value, because value is never an input, but they fragment the anonymity set across thirteen pools. STRK20 keeps a unified set, but replaces set-based anonymity with **address pseudorandomness**: confidentiality holds as long as the channel key stays secret.

This is the observation that Section 4 builds on. Because note\_id derives from channel\_key, that one secret governs both confidentiality **and** addressing. The security properties are coupled in a way they are not in a commitment-tree design.

## 3.6 Accumulator

### STRK20

None. The functional analogue of a historical-roots window is the proof validity window (450 blocks, ~15 min) anchored to a base block, plus ten-block note maturation.

### PROTOCOL 01

Poseidon-Goldilocks Merkle tree, depth 15, 100 historical roots, on-chain subtree maintenance via a dedicated circuit.

These are not equivalent mechanisms: one resolves concurrency on tree state, the other resolves freshness of a virtual execution.

## 3.7 Nullifiers

|                  | STRK20                                                          | PROTOCOL 01                          |
|------------------|-----------------------------------------------------------------|--------------------------------------|
| Formula          | Poseidon(TAG, channel_key, token, index, 0, owner_private_key)  | Poseidon(nullifier_preimage, secret) |
| Storage          | Map<felt252, bool>, WriteOnce                                   | program-owned account                |
| Notable property | the sender cannot compute it, and so cannot watch for the spend | —                                    |

The sender/recipient asymmetry is a property STRK20 has and Protocol 01 does not.

The central axis, and the one where the two designs are most precisely complementary.

|                    | STRK20                                                           | PROTOCOL 01 (SPECTER)                                |
|--------------------|------------------------------------------------------------------|------------------------------------------------------|
| Mechanism          | per-recipient bucket, keyed by <b>plaintext Starknet address</b> | derived <b>stealth address</b> , public announcement |
| Prerequisite       | <b>recipient registered on-chain with a public viewing key</b>   | <b>none</b> — the recipient acts only after the fact |
| Scan cost          | <b>proportional to the user's own activity</b>                   | <b>proportional to total pool volume</b>             |
| Cryptography       | ECDH on the STARK curve + Poseidon                               | <b>hybrid X25519 + ML-KEM-768, OR-secure</b>         |
| Quantum resistance | <b>no</b>                                                        | <b>yes for discovery; no for the spending path</b>   |
| Recipient exposure | <b>address revealed at channel opening</b>                       | <b>no recipient address on-chain</b>                 |
| Ciphertext         | field elements in contract storage                               | 1088 B, chunked into an account                      |
| Maturity           | <b>mainnet</b>                                                   | <b>devnet</b>                                        |

## The complementarity, stated precisely

A stealth-address derivation removes the channel-open linkability that STRK20 documents as a known limitation. A per-recipient bucket removes the pool-volume scan cost that a public-announcement design incurs. Each architecture addresses the other's structural weakness.

Neither is dominant. STRK20 is stronger on scan scalability and on maturity. Specter is stronger on recipient exposure, on not requiring recipient pre-registration, and on quantum resistance of the key exchange. A design combining bucketed delivery with a post-quantum, address-free key agreement would strictly dominate both, and is the direction Section 5 proposes.

## 3.8 Two further axes, briefly

|                 | STRK20                                                                                                                     | PROTOCOL 01                                                                                                        |
|-----------------|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Note encryption | ECDH on the STARK curve, Poseidon KDF, additive mask. <b>Fixed in the contract, no version field</b>                       | X25519 + ML-KEM-768, HKDF-SHA256, explicit version byte in the wire format                                         |
| Viewing keys    | <b>native and mandatory; on-chain escrow to a designated auditor; on-chain deposit screening</b>                           | <b>none</b> — <b>no compliance layer exists</b>                                                                    |
| Account model   | <b>native account abstraction; multisig, hardware and smart accounts work unchanged; a PQ signer is already deployable</b> | Ed25519 and program-derived accounts; a WOTS+ client module exists but the on-chain co-signature verifier does not |
| Composability   | <b>private swaps live on AVNU and Ekubo; anonymizer contract pattern</b>                                                   | <b>to be built</b>                                                                                                 |

On compliance, composability, maturity and post-quantum **signatures**, STRK20 is ahead. Its account abstraction already permits a Falcon-512 signer deployed on mainnet by a third party at roughly 9.5M L2 gas, with no protocol change required. The gap analysed in Section 4 concerns key **encapsulation**, not signatures.



The twelve axes at a glance. Neither column is a scorecard: several rows describe deliberate trade-offs rather than quality differences.

| #  | AXIS                        | STRK20                                                                                   | PROTOCOL 01                                                                     |
|----|-----------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 1  | <b>Proof system</b>         | Circle STARK over M31, blowup 2, 70 queries, 96 bits conjectured                         | Compact STARK over Goldilocks, blowup 16, 27/22 queries, 124/104 bits classical |
| 2  | <b>Constraint authoring</b> | one fixed CPU AIR; the program is data; no circuit DSL                                   | one hand-written AIR per circuit; four synchronised sites per change            |
| 3  | <b>Verification site</b>    | Rust verifier in the sequencer; no compute ceiling; phase-1 security explicitly degraded | on-chain program; hard 1.4M CU per transaction; full L1 security                |
| 4  | <b>Proof transport</b>      | protocol fields on Invoke V3; ~500 KB–1.3 MB class                                       | 80–144 KB padded to 145 KB; 159 transactions; rent locked                       |
| 5  | <b>Note model</b>           | two-field struct; identity is the storage address                                        | Poseidon commitment; identity is a tree leaf                                    |
| 6  | <b>Accumulator</b>          | none; proof validity window instead                                                      | depth-15 Merkle tree, 100 historical roots                                      |
| 7  | <b>Nullifiers</b>           | owner-keyed; the sender cannot observe the spend                                         | secret-keyed                                                                    |
| 8  | <b>Recipient discovery</b>  | bucketed; scan cost independent of pool volume                                           | stealth address; no recipient exposure, no pre-registration                     |
| 9  | <b>Note encryption</b>      | ECDH on curve; fixed in the contract; no version field                                   | hybrid X25519 + ML-KEM-768; versioned wire format                               |
| 10 | <b>Compliance</b>           | viewing keys, auditor escrow, on-chain screening, Lean 4 proofs                          | none                                                                            |
| 11 | <b>Account model</b>        | native AA; PQ signer already deployable                                                  | Ed25519 / PDAs; PQ spend path incomplete                                        |
| 12 | <b>Maturity</b>             | mainnet since April–May 2026; OpenZeppelin audited                                       | devnet; internal soundness audit only                                           |

#### Where STRK20 leads

Constraint authoring, scan scalability, compliance, DeFi composability, account abstraction, post-quantum signatures, formal verification, and production maturity. Most of these follow from owning the L2 and from the fixed-CPU-AIR model.

#### Where Protocol 01 leads

Proof compactness, verification without additional trust assumptions, recipient exposure, absence of recipient pre-registration, and post-quantum key encapsulation. The last of these is the subject of Section 4.

**The claim, stated precisely**

StarkWare's 30 June 2026 post-quantum roadmap scopes itself to hashing and commitments: Pedersen replaced by BLAKE2 across state commitment, contract address derivation and the OS config hash, followed by legacy-contract migration and Ethereum-dependent bridge and data-availability work. It does not address how the encryption layer already shipped in STRK20 — encrypted balances, and the on-chain encrypted viewing key a designated auditor can decrypt — would itself migrate to post-quantum primitives. The proving system is hash-based and survives Shor. The confidentiality layer shipped roughly two months before the roadmap and is not covered by it.

**The three phases**

1 — hash migrations (state commitment, address derivation, OS config), Pedersen→BLAKE2, ~2 months. 2 — storage of existing contracts. 3 — Ethereum dependencies (secp256k1/r1 syscalls, KZG-anchored blobs), blocked on Ethereum

**Falcon-512**

cited for **consensus signatures only**, and as a non-binding example ("such as"). No SNIP has been assigned for Falcon-512 or for post-quantum account signatures as of this writing; SNIP-6 remains the scheme-agnostic account interface standard, at status Review

**Accounts**

covered indirectly: "Starknet users can deploy a PQ wallet today", with no protocol change

**Two concrete structures**

Both are curve-based encryption, published on-chain, permanent. Both are outside the roadmap's stated scope.

**STRUCTURE A****Viewing-key escrow (EncPrivateKey)**

```
enc_private_key = h(ENC_PRIVATE_KEY_TAG, rK.x) + private_key
```

- it protects a user's **entire bidirectional history**, not a single transaction
- it is permanent and immutable by design, so exposure never decays
- a single compromised or broken auditor key affects **every registered user at once**

**STRUCTURE B****Channel opening (EncChannelInfo)**

```
enc_channel_key = h(CHANNEL_KEY_TAG, rK.x) + channel_key
enc_sender_addr = h(SENDER_ADDR_TAG, rK.x) + sender_addr
```

This is where the storage-address design has a consequence that a commitment-tree design does not. Because `note_id = Poseidon(NOTE_ID_TAG, channel_key, token, index, 0)`, recovering a channel key does not merely reveal amounts — it makes every note address in that channel computable. **Confidentiality and addressing rest on the same secret.**

**Why the remedy is narrow**

Additive masking in a prime field has no structure Shor exploits **in itself**. The entire vulnerability sits in the ECDH that produces `rK.x`. Replacing that one source of key material leaves the note structure, nullifiers, discovery walk and phase ordering untouched. Note also what already protects the system: Stwo is hash-based, BLAKE2 is partially deployed, and account abstraction admits a PQ signer today. The missing piece is key encapsulation specifically.

A hybrid key-encapsulation substitution inside STRK20's two encrypted structures, leaving the rest of the protocol unchanged.

## The proposal

Replace the STARK-curve ECDH that produces  $rK.x$  with a hybrid construction whose shared secret is derived from an ML-KEM-768 decapsulation combined with the existing curve secret, reduced into the STARK field. Every downstream consumer continues to receive a  $fe1t252$  of key material.

### Why the scope is small:

- The substitution point is a **single function**. `_compute_shared_x` produces the key material; nothing else in the protocol inspects how it was produced. **Note structure, nullifiers, the discovery walk and the phase ordering are unaffected.**
- The OR-secure combiner already exists and is tested in production code: `HKDF(classic || pq, info = transcript)`. Transposing it to Poseidon keeps the construction inside STRK20's existing hash domain. Security holds if **either** primitive holds, so the change is not a bet on lattices replacing curves.
- No new note system is introduced. The proposal deliberately sits above the pool, not beside it.

## Two implementation constraints worth recording

First, ciphertexts belong in **contract storage, not events**. STRK20 places no encrypted payload in events, and its entire discovery path reads storage; an event-based announcement would diverge from that pattern without benefit. Second, packing matters: a 1088-byte ML-KEM-768 ciphertext occupies 36 field elements at 31 bytes each, against 1088 at one byte each — a thirty-fold difference in cost.

## Open questions this proposal does not resolve

- **Migration.** There is no version field on `Note` or `EncChannelInfo`, and viewing keys are `WriteOnce` and immutable. How existing channels and already-escrowed keys would migrate under any encryption change is not documented, and is a prerequisite question rather than an implementation detail.
- **Ciphertext size.** A 1088-byte ciphertext per channel opening is materially larger than a single field element. The cost is bounded — it is paid once per (sender, recipient) pair, not per note — but it is not free.
- **Auditor key.** If the escrow encryption becomes hybrid, the auditor key itself must gain an ML-KEM component, which interacts with whatever threshold scheme is intended.

## Prior art this builds on

The hybrid construction, the deterministic KEM key derivation from a wallet seed, the chunked on-chain ciphertext transport and the OR-secure combiner are implemented and published as `@protocol-01/specter-sdk`. The cryptographic core is chain-agnostic: the ML-KEM and hashing modules carry no chain dependency, and only the transport and scanning layers are Solana-specific. Porting the construction is an adapter exercise, not a cryptographic one.

### STRK20 and Starknet

- [github.com/starkware-libs/starknet-privacy](https://github.com/starkware-libs/starknet-privacy) — Cairo pool contract, Rust discovery service, TypeScript SDK, Lean 4 proofs. Apache 2.0
- [community.starknet.io](https://community.starknet.io) — SNIP-36, in-protocol proof verification
- [starkware.co/blog](https://starkware.co/blog) — post-quantum roadmap, 30 June 2026
- [github.com/starkware-libs/stwo](https://github.com/starkware-libs/stwo) and [stwo-cairo](https://github.com/starkware-libs/stwo-cairo)
- ePrint 2024/278 (Circle STARKs), ePrint 2021/1063 (Cairo)
- [docs.starknet.io](https://docs.starknet.io) — fees, S-two book, benchmarks
- [@starknet-io/types-js@0.10.3](https://@starknet-io/types-js@0.10.3) — Wallet API v0.10.3
- [openzeppelin.com/news/privacy-contracts-audit](https://openzeppelin.com/news/privacy-contracts-audit)
- [strk20-by-example.org](https://strk20-by-example.org) — community documentation, not an official property, but quotes the Cairo source verbatim and matched it on every cross-checked claim

### Protocol 01

- [stark/src/compact.rs](https://stark/src/compact.rs) — the proof protocol verified on-chain
- [stark/src/air/](https://stark/src/air/) — the seven circuit AIRs
- [programs/p01\\_stark\\_verifier/](https://programs/p01_stark_verifier/) — on-chain verifier, proof parsing, periodic constants
- [programs/zk\\_shielded/](https://programs/zk_shielded/) — shielded pool and Merkle state
- [packages/specter-sdk](https://packages/specter-sdk) — recipient discovery, published as [@protocol-01/specter-sdk](https://@protocol-01/specter-sdk)
- [programs/specter/](https://programs/specter/) — announcement transport

### Verification caveats

Recorded so that readers can weigh each claim appropriately.

- The observation that the post-quantum roadmap does not mention encryption or key encapsulation was established by targeted term checks through a summarising retrieval tool rather than a raw full-text search. It should be confirmed directly against the source document before being relied upon.
- Protocol 01's compute-unit consumption is **not measured anywhere in its repository**. Only an assertion that each phase stays below the per-transaction limit exists. No compute-unit figure is quoted in this document.
- STRK20's published proving time has two conflicting values (~29 s and ~4 s) that no source reconciles.
- STRK20's per-transaction proof size is not published; only SNIP-36's generic cost model is available.
- The mainnet auditor identity, the current auditor public key, and whether a threshold scheme is deployed are not public.
- Public documentation names FPI as the deposit screening provider while the repository calls Elliptic's API; the relationship between the two is not documented.

This document is a technical comparison prepared for the STRK20 buildathon, Paris, 22 July 2026. It is not an audit, and it makes no security claim about either system. Findings attributed to the OpenZeppelin audit are drawn from the public summary page rather than the full report.